

Selecting Securities (Users)

Due to the specialness of securities (commodities), both on the technical level and on the business-logic level, there are several things you must understand/keep in mind before working with them.

In a nutshell, it boils down to the following: There normally are technical IDs, and there are business logic IDs. And sometimes, as here, there are pseudo-technical IDs, which in fact are business-logic IDs.

Please read the document “ID Layers in GnuCash” (folder `xsec`) first before you move on. It is important to understand the (non-)difference between GnuCash’s technical and business-logic security IDs described there before reading the next section; otherwise it will probably confuse you.

Using the Tools

We use the test data file provided with module “API Extensions”.¹ For test and illustration purposes, we have put securities into this file using various different systems (i.e., name spaces) – something you normally would not do in real life.

There are several tools where you will have to specify a security, but only one of them supports all variants that we will cover in this section: **GetSec**. We have provided a wrapper shell script for it: `gcsh_get_sec_info.sh`.

Then, you have several options:

- *Mode “ID”*: Specifying the security by its (pseudo-)technical ID.

This variant has two sub-variants:

- *Sub-mode “DIRECT”*: Specifying the (pseudo-)technical ID directly, using the syntax `<namespace>:<code>`.

This approach always works, but does not leverage the pre-defined name spaces. In case of an error, you won’t get precise logs and could possibly spend a lot of time finding it.

Example:

```
$ gcsh_get_sec_info.sh \  
  -f test.gnucash \  
  -ssm ID \  
  -sssm DIRECT \  
  -sec EURONEXT:SAP  
  
or  
  
$ gcsh_get_sec_info.sh \  
  -f test.gnucash \  
  -ssm ID \  
  -sssm DIRECT \  
  -sec ISIN:DE000BASF111AP
```

¹The test data files of the other modules will almost certainly work as well – they all are very similar.

In essence, with the direct method, we kind of “stubbornly” pretend not to know that the “technical” ID is, in fact, a business-logic ID. Instead, we see the ID as just an unnecessarily long string that has no further meaning.

The advantage of this “stubborn” approach is that you can use all provided tools somewhat symmetrically (remember that all other entities in GnuCash do have genuine technical IDs).

- *Sub-mode “INDIRECT_XYZ”*: Specifying the (pseudo-)technical ID indirectly, using the various command line options.

This approach leverages the `JGnuCashLib`’s built-in type-safety early in the execution and will thus lead to far more precise error logs in case of an error.

There are three sub-modes to choose from:

* `INDIRECT_EXCHANGE_TICKER`:

Here, `<namespace>:<code>` becomes `<exchange:ticker>`, where `<exchange>` is the semi-formal, non-standardized but widely-used abbreviation of a major exchange (e.g., “NYSE”, “EURONEXT”, “XETRA”, “JPX”, etc.), and `<ticker>` is the exchange-specific ticker of the security.

* `INDIRECT_MIC`:

Here, `<namespace>:<code>` becomes `<mic:mic-code>`, where `<mic>` is the formal Market Identifier Code (MIC), which has been standardized in ISO 10383 (e.g., “XNYS”, “XPAR”, “XSHE” etc.), and `<mic-code>` is the same as `<ticker>` above.

* `INDIRECT_SEC_ID_TYPE`:

Here, `<namespace>:<code>` becomes `<sec-id-type:sec-id>`, where `<sec-id-type>` is the type of the public security ID (e.g. “ISIN”, “CUSIP”, “SEDOL”, “WKN” or similar), and `<sec-id>` is the actual security ID under the security ID type’s system (e.g., “DE0007100000” for “ISIN”, “710000” for “WKN”, etc.).

Example:

```
$ gcsh_get_sec_info.sh \
  -f test.gnucash \
  -ssm ID \
  -sssm INDIRECT_EXCHANGE_TICKER \
  -exch EURONEXT \
  -tkr SAP
```

or

```
$ gcsh_get_sec_info.sh \
  -f test.gnucash \
  -ssm ID \
  -sssm INDIRECT_SEC_ID_TYPE \
  -secid-type ISIN \
  -is DE000BASF111AP
```

In essence, with the indirect method, we “acknowledge” the semantics of the pseudo-technical ID, as opposed to the direct method.

Notice that the second example uses the ISIN, which in this case is used to specify the security's (pseudo-)technical ID (because the parameter `-ssm` is set to "INDIRECT_SEC_ID_TYPE").

Also notice the parameters `-exch Euronext` and `-secid-type ISIN` in the examples. These are, as already stated, *pre-defined* values of `JGnuCashLib`, and strictly speaking, they have nothing to do with what you actually have in your GnuCash file (obviously, it is strongly recommended that you use these pre-defined values when editing securities in GnuCash).² But if, for example, in one of your file's securities you wrote, say, "EURONXT" instead of "EURONEXT" by accident, then you would not be able to retrieve that one with the above-mentioned indirect method. You would instead have to use the direct method.

On the other hand, you don't need to make this kind of mistake: Just use the tool `GenSec`, and `JGnuCashLib` will ensure that only the pre-defined values are used.

- *Mode "ISIN"*: Specifying the security by its business logic ID (the field "X-Code" in GnuCash lingo), i.e. its ISIN, CUSIP, SEDOL, WKN or similar official security identifiers (without the name-space prefix).

This approach obviously only works when you actually have filled the field "X-Code", which is typically redundant to the security's pseudo-technical ID. That bears potential for errors, so this approach will not always work. However, once you have your data in order, it is the most convenient method.

Example:

```
$ gcmsh_get_sec_info.sh \  
  -f test.gnucash \  
  -ssm ISIN \  
  -is DE000BASF111
```

Notice the difference to the previous example: We also use the parameter `-is` here, but this time, it is used to search over the field "X-Code" (because the parameter `-ssm` is set to "ISIN"). Also notice that – for the time being – the exact same command is used when you have CUSIPs, SEDOLs, WKNs or something else in your X-Code. Still the above notation with "ISIN" etc. is used.³

- *Mode "NAME"*: Specifying the security by its name.

This method is generally *not* recommended, but the maintainer acknowledges that there might be special use cases where it is appropriate. (Apart from that, this method only works for `GetSec` anyway, not for `UpdSec`.)

Example:

```
$ gcmsh_get_sec_info.sh \  
  -f test.gnucash \  
  -ssm NAME \  
  -sn "Mercedes-Benz Group AG"
```

²To get a list of all these values, just start the program with the help flag: `gcmsh_get_sec_info.sh -h`.

³The current maintainer uses only ISINs in his own business' file, because the ISIN is the only truly global and stable identifier.

Keep in mind, though, that the name has to be specified *completely* and *excatly* as it is stored in the GnuCash file – it is case sensitive, has zero tolerance for leading or trailing empty spaces, etc. Thus, the following will *not* work:

- `-sn "Merced"`
- `-sn "mercedes-benz group"`
- `-sn "Mercedes Benz"`

You can, however, use the tool **GetSecList** to search for a security the name and exact spelling of which you do not know (i.e., what one usually would expect). This tool is tolerant against upper- vs. lower-case spelling, will accept only parts of the name, etc. With it, you can get the (list of) matching security/ies (short info), and with this info (ID, X-Code), you can use one of the other methods above to get the security's full information.